O que você está propondo é mais amplo do que uma ferramenta de análise de aplicativos. É a criação de uma plataforma SaaS de inteligência de mercado mobile, aquisição de usuários, publicidade e monetização de aplicativos.

A ideia é interessante porque une dois mercados que normalmente são atendidos por plataformas diferentes:

1. Inteligência de aplicativos: descobrir oportunidades, analisar concorrentes, palavras-chave, downloads, rankings e tendências.
2. Publicidade e monetização: permitir que desenvolvedores divulguem seus aplicativos, comprem tráfego, monetizem espaços publicitários e troquem anúncios entre aplicativos.

Eu estruturaria o projeto como um ecossistema com quatro grandes produtos integrados.

1. Visão geral da plataforma

Plataforma central

Mobile Growth Platform

Inteligência • Aquisição • Publicidade • Monetização

App Intelligence

ASO, rankings, palavras-chave, concorrentes e tendências.

Ad Network

Rede de anúncios, campanhas pagas e troca de publicidade.

App Growth

Aquisição, instalação, campanhas e acompanhamento de usuários.

Monetization Hub

Receita publicitária, inventário, créditos e monetização cruzada.

Núcleo central de dados

Apps • Desenvolvedores • Audiências • Campanhas • Impressões • Cliques • Instalações • Receita

2. Módulo de inteligência de aplicativos

Referências: AppBrain, AppTweak e MobileAction.

Este módulo seria responsável por ajudar os desenvolvedores a descobrir o que criar, como posicionar seus aplicativos e como superar concorrentes.

AppTweak - App Store Marketing & Intelligence Platform
Android App Rankings - Statistics by AppBrain
9

Funcionalidades

Recurso	Funcionalidade
App Rankings	Ranking de aplicativos por país e categoria
Keyword Research	Pesquisa de palavras-chave
Keyword Tracking	Monitoramento de posições
Competitor Intelligence	Análise de concorrentes
App Discovery	Descoberta de aplicativos novos
Download Estimates	Estimativa de downloads
Revenue Estimates	Estimativa de faturamento
Category Trends	Tendências por categoria
App History	Histórico de evolução
ASO Audit	Auditoria de páginas de aplicativos

Diferencial importante

O sistema deve permitir pesquisar, por exemplo:

- Aplicativos de futebol no Brasil.
- Aplicativos de inteligência artificial crescendo.
- Aplicativos de limpeza de celular.
- Aplicativos com mais de 100 mil downloads.
- Aplicativos com muitas avaliações negativas.
- Aplicativos com crescimento recente.
- Palavras-chave com demanda e baixa concorrência.

Atenção aos dados: a Google Play não fornece publicamente o volume exato de pesquisas internas nem downloads individuais em tempo real. A plataforma deverá trabalhar com estimativas, dados públicos, fontes licenciadas e dados fornecidos pelos próprios desenvolvedores. Não devemos apresentar estimativas como números oficiais.

3. Módulo de publicidade: sua própria Ad Network

Este é um dos componentes mais importantes do projeto.

A proposta é criar uma rede publicitária própria, na qual desenvolvedores possam oferecer espaços de publicidade e anunciantes possam comprar impressões, cliques ou instalações.

Referências conceituais: Google AdMob, Unity Ads e redes de cross-promotion.

User acquisition campaign setup • Unity Grow • Unity Docs
PubScale: One-stop growth and monetization platform
Yayıncının kontrol listesi: Mobil uygulamanızda reklamlarla nasıl para kazanırsınız? - ShiftDelete.Net
9

3.1. Dois tipos de publicidade

A. Publicidade paga

O desenvolvedor ou anunciante compra espaço publicitário para divulgar seu aplicativo.

Exemplo:

- Aplicativo A compra uma campanha.
- Escolhe Brasil como público.
- Define Android ou iOS.
- Seleciona categorias e interesses.
- Define orçamento de R$ 500.
- Escolhe banners, anúncios nativos ou interstitial.
- A plataforma distribui os anúncios nos aplicativos participantes.

O anunciante acompanha impressões, cliques, CTR, instalações atribuídas e custo por resultado, conforme a capacidade de mensuração disponível.

B. Troca de anúncios (Ad Exchange / Cross-Promotion)

Aqui está uma possibilidade interessante para desenvolvedores que não possuem orçamento de publicidade.

Exemplo:

Exemplo de troca de publicidade

FlashScore – Live Sports Score App by RonDesignLab on Dribbble

App de Futebol

Oferece anúncios

10.000

Impressões disponibilizadas

Homepage Design - Edlino Gamified Language Learning App by Altamira Tripty for Netro Systems on Dribbble

App de Idiomas

Recebe anúncios

10.000

Impressões recebidas

Troca de inventário

Exemplo ilustrativo, não uma equivalência automática de valor econômico.

O aplicativo de futebol divulga o aplicativo de idiomas e vice-versa.

Mas eu implementaria um sistema de créditos, em vez de uma troca obrigatoriamente 1:1.

Por exemplo:

- O desenvolvedor disponibiliza 10.000 impressões.
- Recebe créditos conforme impressões válidas, qualidade e valor do inventário.
- Utiliza os créditos para divulgar seu próprio aplicativo.
- Pode comprar créditos adicionais com dinheiro.
- Pode também vender inventário para receber receita financeira.

Isso cria uma economia interna.

3.2. Três modalidades de campanhas

Modalidade	Objetivo
CPC	Pagar por clique
CPM	Pagar por mil impressões válidas
CPI	Pagar por instalação atribuída

Eu começaria com CPM e CPC. O CPI exigiria uma camada mais robusta de atribuição de instalações, prevenção de fraude e integração com ferramentas de mensuração mobile.

3.3. Formatos de anúncios

Lokal Banner Advertisement | Lokal Banner Ad Cost

Banner

Publicidade fixa no aplicativo.

기본 설정 | Buzzvil Docs

Native Ads

Anúncio integrado ao conteúdo.

InMobi Support Center for Advertisers and Publishers

Interstitial

Publicidade em tela cheia.

Effective app monetization: The all-in-one guide | Adjust

Rewarded Ads

Usuário assiste e recebe recompensa.

4. Módulo House Ads

Este módulo merece atenção especial porque pode ser uma das primeiras funcionalidades comercialmente úteis.

House Ads são anúncios próprios, utilizados para promover aplicativos, produtos e serviços sem necessariamente pagar uma rede externa.

Imagine que você tenha dez aplicativos publicados.

Sua plataforma poderia permitir:

- Promover o aplicativo de futebol no aplicativo de idiomas.
- Promover o aplicativo de limpeza no aplicativo de futebol.
- Divulgar novos lançamentos em todos os aplicativos.
- Fazer campanhas segmentadas por categoria.
- Direcionar tráfego para seus próprios aplicativos.

Manage Ad – Developer guides
developers.withhive.com

Eu criaria três níveis:

1. House Ads próprios: divulgação gratuita dos aplicativos do proprietário da plataforma.
2. Cross-Promotion: troca de anúncios entre desenvolvedores.
3. Paid Ads: campanhas comerciais pagas por anunciantes.

O mesmo mecanismo de distribuição pode atender aos três, mas com regras diferentes de faturamento e prioridade.

5. Módulo de monetização

O objetivo é permitir que o desenvolvedor escolha como deseja monetizar seu inventário publicitário.

Modelo	Como funciona
Monetização financeira	Recebe dinheiro pelas impressões
Troca de anúncios	Recebe créditos publicitários
Monetização híbrida	Parte em dinheiro e parte em créditos
House Ads	Divulga seus próprios aplicativos
Campanhas patrocinadas	Recebe para divulgar marcas e produtos

Exemplo de configuração de um aplicativo

O desenvolvedor cadastra seu aplicativo e configura:

Monetização do aplicativo

Publicidade paga

Receber receita financeira

Ativado

Troca de anúncios

Acumular créditos

Ativado

House Ads

Divulgar aplicativos próprios

Ativado

Distribuição do inventário

Exemplo de configuração editável conceitualmente pelo desenvolvedor.

Receita financeira

50%

Créditos de troca

30%

House Ads

20%

Essa divisão é apenas um exemplo. O sistema deverá contabilizar impressões efetivamente válidas, não apenas a quantidade de espaços disponibilizados.

6. Painel do desenvolvedor

O desenvolvedor terá uma única conta para administrar todos os seus aplicativos.

| Magify
magify.com

Funcionalidades do painel

- Cadastro de aplicativos Android e iOS.
- Integração com Google Play e App Store.
- Análise de downloads e avaliações.
- Monitoramento de palavras-chave.
- Comparação com concorrentes.
- Gestão de campanhas publicitárias.
- Gestão de House Ads.
- Controle de créditos.
- Relatório de receita.
- Relatório de impressões e cliques.
- Gestão de espaços publicitários.
- Configuração de formatos e frequência dos anúncios.

Também criaria dois ambientes:

Publisher: quem disponibiliza espaços publicitários nos seus aplicativos.

Advertiser: quem compra publicidade para divulgar seus aplicativos.

Uma mesma conta pode exercer os dois papéis.

7. Arquitetura técnica

Minha recomendação é construir como uma plataforma SaaS modular, preparada para crescer sem precisar reescrever tudo quando a rede de publicidade ganhar volume.

Frontend

Dashboard Web • Administração • API pública

Next.js / React

Backend — NestJS

App Intelligence

ASO Engine

Ad Campaigns

Ad Exchange

Ad Server

House Ads

Billing & Wallet

Analytics

PostgreSQL

Dados e transações

Redis

Cache e filas

Workers

Coleta e processamento

Analytics

Eventos e métricas

SDK de publicidade

Flutter • Android nativo • iOS

Integração com aplicativos dos publishers

Aplicativos participantes

Impressões • Cliques • Campanhas • Receita • Créditos

Tecnologias recomendadas

Componente	Tecnologia
Dashboard	Next.js
Backend	NestJS
Banco principal	PostgreSQL
Cache e filas	Redis
Processamento	Workers NestJS
Analytics de eventos	ClickHouse
SDK mobile	Flutter + Android/iOS nativo quando necessário
Infraestrutura	Hostinger VPS + EasyPanel
Deploy	GitHub + EasyPanel

Eu manteria a arquitetura modular, com módulos independentes dentro do NestJS inicialmente. Não começaria com dezenas de microsserviços.

8. Como obter os dados de inteligência

Este é um dos desafios técnicos mais importantes.

Fonte	Dados possíveis
Google Play	Aplicativos, descrições, categorias, avaliações, rankings públicos
App Store	Metadados, categorias e rankings públicos
Google Play Console	Dados privados dos aplicativos autorizados pelo desenvolvedor
App Store Connect	Dados privados dos aplicativos autorizados
Google Trends	Interesse relativo por assunto
Ferramentas ASO	Estimativas de palavras-chave e concorrência
SDK próprio	Impressões, cliques e comportamento publicitário
Dados dos clientes	Downloads, receita e conversões autorizadas

A vantagem competitiva pode vir da combinação dessas fontes.

Por exemplo, você poderá identificar um aplicativo crescendo na categoria de futebol e oferecer ao desenvolvedor uma campanha para atingir usuários interessados em futebol.

Não dependeria exclusivamente de scraping. Usaria APIs oficiais, fontes licenciadas, dados públicos permitidos e integrações autorizadas.

9. Como a plataforma ganha dinheiro

Você terá várias fontes de receita independentes.

Fonte	Modelo de cobrança
ASO Premium	Assinatura mensal
Inteligência de mercado	Planos por volume de consultas

Publicidade paga	Margem sobre campanhas
Ad Network	Comissão sobre inventário
Créditos de publicidade	Venda de pacotes
Analytics Premium	Assinatura
API comercial	Cobrança por requisição ou plano
Gestão de campanhas	Planos para agências

Exemplo de planos comerciais

Valores iniciais hipotéticos, para validar o posicionamento:

Plano	Preço mensal
Free	R$ 0
Starter	R$ 49
Pro	R$ 149
Business	R$ 399
Agency	R$ 799

A rede publicitária teria cobrança separada por orçamento de campanha.

10. Um cuidado fundamental: fraude e qualidade do tráfego

Uma rede própria de anúncios precisa nascer com mecanismos antifraude.

Alguns exemplos:

- Cliques repetidos ou automatizados.
- Instalações falsas.
- Impressões invisíveis.
- Aplicativos com tráfego artificial.
- Publishers que manipulam eventos.
- Trocas de anúncios sem usuários reais.
- Fazendas de dispositivos.

Eu criaria desde o começo um sistema de validação de eventos, limites por dispositivo, identificação de tráfego suspeito, auditoria de publishers e retenção de créditos até a validação.

Também precisaremos de regras claras para consentimento, privacidade, LGPD, políticas das lojas e identificação dos anúncios.

Importante: créditos de troca não devem ser tratados como receita financeira ou instalações garantidas. E a rede deve ser independente do inventário do AdMob: não se

deve usar cliques ou impressões artificiais, nem incentivar cliques em anúncios de terceiros.

11. Plano de desenvolvimento por fases

Eu não desenvolveria tudo ao mesmo tempo. A inteligência de mercado e a rede de anúncios são produtos grandes, com desafios diferentes.

Minha sugestão:

1. Fase 1 — App Intelligence MVP
    - Cadastro e pesquisa de aplicativos.
    - Rankings por país e categoria.
    - Pesquisa de palavras-chave.
    - Análise de concorrentes.
    - Histórico de rankings.
    - Painel do desenvolvedor.
2. Fase 2 — House Ads
    - Cadastro de inventário.
    - SDK Flutter inicial.
    - Banners e anúncios nativos.
    - Campanhas próprias.
    - Relatórios de impressões e cliques.

    Essa fase permite utilizar seus próprios aplicativos como primeiros publishers.

3. Fase 3 — Rede de troca de anúncios
    - Cadastro de publishers e advertisers.
    - Carteira de créditos.
    - Regras de conversão de impressões em créditos.
    - Campanhas de cross-promotion.
    - Sistema de distribuição e balanceamento.
4. Fase 4 — Rede de publicidade paga
    - Compra de campanhas.
    - Orçamentos e pagamentos.
    - CPM e CPC.
    - Segmentação.
    - Relatórios financeiros.
    - Repasse aos publishers.
5. Fase 5 — Inteligência avançada
    - Estimativas de downloads e receita.
    - Monitoramento de tendências.
    - Recomendações de palavras-chave.
    - Análise competitiva avançada.
    - API comercial.
    - Recomendações de campanhas com IA.

12. Oportunidade estratégica

Existe uma combinação especialmente interessante nessa ideia:

Um desenvolvedor entra na plataforma para descobrir quais aplicativos estão crescendo, utiliza a inteligência para encontrar uma oportunidade, cria seu aplicativo, divulga pela rede de anúncios e depois monetiza seu inventário.

Você passa a atender praticamente todo o ciclo de vida de um aplicativo.

Deep Dive: Ad Mediation Platforms and Why They Matter - AdBreak Media - AdBreak Media
adbreakmedia.com

Mas existe um desafio comercial: uma rede publicitária precisa de anunciantes e publishers ao mesmo tempo. Por isso, começaria com House Ads e troca de anúncios, aproveitando seu próprio portfólio de aplicativos para criar o inventário inicial. Depois abriria a rede para terceiros.

13. Precisamos definir o posicionamento do produto

Minha proposta é tratar isso como uma nova empresa/plataforma SaaS, não como uma simples ferramenta de ASO.

Antes de desenhar o projeto técnico completo, quero definir uma decisão estratégica:

1. Qual será o mercado inicial?

Android e iOS desde o início
Android primeiro, iOS posteriormente
iOS primeiro, Android posteriormente

2. Como funcionará a rede publicitária?

Híbrida: publicidade paga + troca de anúncios + House Ads
Somente troca de anúncios inicialmente
Publicidade paga e monetização financeira

3. Qual será a abrangência do projeto?

Plataforma completa, desenvolvida por fases
Inteligência de aplicativos primeiro
Rede de anúncios primeiro

4. Qual será o mercado geográfico?

Brasil inicialmente
Global desde o início
Brasil primeiro, arquitetura internacional

Definir direcionamento do projeto

Com essas definições, podemos elaborar o projeto executivo, incluindo arquitetura, banco de dados, APIs, SDK Flutter, painel administrativo, motor de anúncios, sistema de créditos e modelo financeiro. A ideia é construir uma plataforma que possa competir em funcionalidades com essas ferramentas, mas com um diferencial próprio: inteligência de mercado integrada a uma rede de aquisição e monetização de aplicativos.

Projeto executivo — Plataforma de Inteligência, Publicidade e Monetização Mobile

Direcionamento estratégico definido

Mobile Growth & Ad Network

Mercado inicial: Brasil
Plataforma: Android
Modelo publicitário: Híbrido — House Ads + troca de anúncios + publicidade paga
Escopo: Plataforma completa, desenvolvida por fases
Modelo de negócio: SaaS + Ad Network + Marketplace de publicidade

Minha recomendação é desenvolver uma plataforma própria, com infraestrutura e tecnologia sob seu controle, que combine as funcionalidades de inteligência de mercado do AppBrain, AppTweak e MobileAction com uma rede de publicidade mobile.

O diferencial não será apenas oferecer ferramentas de análise. Será conectar inteligência, aquisição de usuários e monetização em um único ecossistema.

Vou organizar o projeto como se estivéssemos preparando sua execução com Antigravity, GitHub, Hostinger VPS e EasyPanel.

1. Identidade e posicionamento do produto

O nome de trabalho será Mobile Growth Platform (MGP). O nome comercial definitivo poderá ser escolhido posteriormente.

The best AppSamurai alternative - PubScale
pubscale.com

Proposta de valor

Uma plataforma na qual qualquer desenvolvedor Android poderá:

- Descobrir quais aplicativos estão crescendo no mercado.
- Pesquisar oportunidades de aplicativos.
- Monitorar concorrentes.
- Pesquisar palavras-chave da Google Play.
- Analisar rankings e tendências.
- Divulgar seus aplicativos.
- Comprar campanhas publicitárias.
- Monetizar espaços publicitários.
- Trocar publicidade por créditos.
- Gerenciar toda sua operação em um único painel.

Público-alvo

Público	Necessidade
Desenvolvedores independentes	Descobrir oportunidades e divulgar apps
Pequenas software houses	Aquisição e monetização
Empresas de aplicativos	Inteligência competitiva
Agências de marketing	Gerenciar campanhas
Publishers	Monetizar inventário
Anunciantes	Comprar tráfego mobile

2. Estrutura geral da plataforma

A plataforma terá sete grandes áreas de negócio.

MGP — Core Platform

Identidade, contas, permissões, pagamentos e dados centrais

App Intelligence

Pesquisa e inteligência

ASO Studio

Palavras-chave e otimização

Ad Network

Distribuição de anúncios

Ad Exchange

Troca de publicidade

House Ads

Cross-promotion

Monetization

Receita e créditos

Analytics

Métricas e relatórios

Developer Hub

SDK, API e aplicativos

Infraestrutura compartilhada

PostgreSQL • Redis • ClickHouse • Object Storage • Workers • APIs

2.1. Módulos e responsabilidades

Módulo	Responsabilidade
Core	Usuários, empresas, organizações e permissões
Intelligence	Catálogo e inteligência de aplicativos
ASO	Palavras-chave e otimização
Competitors	Monitoramento de concorrentes
Ad Server	Seleção e entrega de anúncios
Campaign Manager	Gestão de campanhas
Exchange	Mercado de troca de inventário
House Ads	Promoção de aplicativos próprios
Publisher	Gestão dos aplicativos e espaços
Advertiser	Compra de publicidade

Wallet	Carteira de créditos e saldos
Billing	Cobranças, pagamentos e repasses
Analytics	Relatórios e métricas
Fraud Detection	Identificação de tráfego inválido
SDK Manager	Configuração e distribuição do SDK
Admin/NOC	Administração e monitoramento da plataforma

3. App Intelligence — Inteligência de aplicativos

Este será o módulo de pesquisa e análise de mercado.

Introducing the App Intelligence Dashboard: Deep Data for Every App — AppBrain Blog
AppTweak - App Store Marketing & Intelligence Platform
7

3.1. Catálogo de aplicativos

Criar uma base de dados própria de aplicativos da Google Play brasileira.

Cada aplicativo terá:

- Nome.
- Identificador do pacote.
- Desenvolvedor.
- Categoria.
- Descrição.
- Ícone.
- Imagens e capturas de tela.
- Data de publicação, quando disponível.
- Última atualização.
- Nota média.
- Quantidade de avaliações.
- Faixa pública de instalações.
- Países disponíveis.
- Histórico de informações coletadas.

Exemplo:

Aplicativo	Categoria	Instalações públicas
App Futebol A	Esportes	1 mi+
App Limpeza B	Ferramentas	500 mil+
App Idiomas C	Educação	100 mil+

Esses números representam faixas publicadas pela loja, não instalações exatas.

3.2. Ranking de aplicativos

Filtros:

- Brasil.
- Categoria.
- Gratuitos.
- Pagos.
- Novos aplicativos.
- Mais avaliados.
- Melhor avaliação.
- Crescimento estimado.
- Aplicativos atualizados recentemente.

3.3. Pesquisa de palavras-chave

O usuário poderá pesquisar:

`futebol`, `limpar celular`, `inteligência artificial`, `emagrecer`, `editor de fotos`.

Resultados:

- Estimativa de demanda.
- Dificuldade de posicionamento.
- Aplicativos posicionados.
- Tendência histórica.
- Variações da palavra.
- Oportunidades de palavras relacionadas.

Importante: volume de buscas internas da Google Play não é um dado público disponível livremente. O sistema deverá distinguir claramente volume estimado, interesse relativo e sugestões observadas.

3.4. Análise de concorrentes

O usuário adiciona até determinado número de aplicativos para acompanhar.

O sistema monitora:

- Posição nos rankings.
- Nota média.
- Avaliações novas.
- Atualizações.
- Mudanças na descrição.
- Alterações nas capturas de tela.
- Faixas de instalações.
- Evolução de palavras-chave.

3.5. App Opportunity Finder

Esse será um dos recursos mais interessantes.

O usuário seleciona uma categoria e recebe oportunidades classificadas considerando:

- Demanda estimada.
- Crescimento do interesse.
- Concorrência.
- Qualidade dos aplicativos existentes.
- Espaço para diferenciação.
- Potencial de monetização.

O resultado não deve ser uma promessa de sucesso, mas um indicador transparente de oportunidade, com metodologia e fontes.

4. Ad Network — Motor próprio de publicidade

Este será o coração operacional da plataforma e merece uma arquitetura separada dos módulos de inteligência.

A rede terá três canais de distribuição:

Canal	Objetivo	Receita
House Ads	Divulgação própria	Aquisição interna
Ad Exchange	Troca de anúncios	Créditos
Paid Network	Venda de publicidade	Dinheiro

4.1. Como funcionará o fluxo publicitário

#chatgpt-mermaid-_r_rc_{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;fill:rgb(13, 13, 13);}@keyframes edge-animation-frame{from{stroke-dashoffset:0;}}@keyframes dash{to{stroke-dashoffset:0;}}#chatgpt-mermaid-_r_rc_ .edge-animation-slow{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 50s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_rc_ .edge-animation-fast{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 20s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_rc_ .error-icon{fill:rgb(243, 243, 243);}#chatgpt-mermaid-_r_rc_ .error-text{fill:rgb(13, 13, 13);stroke:rgb(13, 13, 13);}#chatgpt-mermaid-_r_rc_ .edge-thickness-normal{stroke-width:1px;}#chatgpt-mermaid-_r_rc_ .edge-thickness-thick{stroke-width:3.5px;}#chatgpt-mermaid-_r_rc_ .edge-pattern-solid{stroke-dasharray:0;}#chatgpt-mermaid-_r_rc_ .edge-thickness-invisible{stroke-width:0;fill:none;}#chatgpt-mermaid-_r_rc_ .edge-pattern-dashed{stroke-dasharray:3;}#chatgpt-mermaid-_r_rc_ .edge-pattern-dotted{stroke-

dasharray:2;}#chatgpt-mermaid-_r_rc_ .marker{fill:rgb(143, 143, 143);stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rc_ .marker.cross{stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rc_ svg{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;}#chatgpt-mermaid-_r_rc_ p{margin:0;}#chatgpt-mermaid-_r_rc_ .label{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";color:rgb(13, 13, 13);}#chatgpt-mermaid-_r_rc_ .cluster-label text{fill:rgb(13, 13, 13);}#chatgpt-mermaid-_r_rc_ .cluster-label span{color:rgb(13, 13, 13);}#chatgpt-mermaid-_r_rc_ .cluster-label span p{background-color:transparent;}#chatgpt-mermaid-_r_rc_ .label text,#chatgpt-mermaid-_r_rc_ span{fill:rgb(13, 13, 13);color:rgb(13, 13, 13);}#chatgpt-mermaid-_r_rc_ .node rect,#chatgpt-mermaid-_r_rc_ .node circle,#chatgpt-mermaid-_r_rc_ .node ellipse,#chatgpt-mermaid-_r_rc_ .node polygon,#chatgpt-mermaid-_r_rc_ .node path{fill:rgb(222, 234, 251);stroke:rgb(83, 154, 248);stroke-width:1px;}#chatgpt-mermaid-_r_rc_ .rough-node .label text,#chatgpt-mermaid-_r_rc_ .node .label text,#chatgpt-mermaid-_r_rc_ .image-shape .label,#chatgpt-mermaid-_r_rc_ .icon-shape .label{text-anchor:middle;}#chatgpt-mermaid-_r_rc_ .node .katex path{fill:#000;stroke:#000;stroke-width:1px;}#chatgpt-mermaid-_r_rc_ .rough-node .label,#chatgpt-mermaid-_r_rc_ .node .label,#chatgpt-mermaid-_r_rc_ .image-shape .label,#chatgpt-mermaid-_r_rc_ .icon-shape .label{text-align:center;}#chatgpt-mermaid-_r_rc_ .node.clickable{cursor:pointer;}#chatgpt-mermaid-_r_rc_ .root .anchor path{fill:rgb(143, 143, 143)!important;stroke-width:0;stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rc_ .arrowheadPath{fill:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rc_ .edgePath .path{stroke:rgb(143, 143, 143);stroke-width:1px;}#chatgpt-mermaid-_r_rc_ .flowchart-link{stroke:rgb(143, 143, 143);fill:none;}#chatgpt-mermaid-_r_rc_ .edgeLabel{background-color:rgb(252, 252, 252);text-align:center;}#chatgpt-mermaid-_r_rc_ .edgeLabel p{background-color:rgb(252, 252, 252);}#chatgpt-mermaid-_r_rc_ .edgeLabel rect{opacity:0.5;background-color:rgb(252, 252, 252);fill:rgb(252, 252, 252);}#chatgpt-mermaid-_r_rc_ .labelBkg{background-color:rgba(252, 252, 252, 0.5);}#chatgpt-mermaid-_r_rc_ .cluster rect{fill:rgb(243, 243, 243);stroke:rgba(0, 0, 0, 0.1);stroke-width:1px;}#chatgpt-mermaid-_r_rc_ .cluster text{fill:rgb(13, 13, 13);}#chatgpt-mermaid-_r_rc_ .cluster span{color:rgb(13, 13, 13);}#chatgpt-mermaid-_r_rc_ div.mermaidTooltip{position:absolute;text-align:center;max-width:200px;padding:2px;font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:12px;background:rgb(243, 243, 243);border:1px solid rgba(0, 0, 0, 0.1);border-radius:2px;pointer-events:none;z-index:100;}#chatgpt-mermaid-_r_rc_ .flowchartTitleText{text-anchor:middle;font-size:18px;fill:rgb(13, 13, 13);}#chatgpt-mermaid-_r_rc_ rect.text{fill:none;stroke-width:0;}#chatgpt-mermaid-_r_rc_ .icon-shape,#chatgpt-mermaid-_r_rc_ .image-shape{background-color:rgb(252, 252, 252);text-align:center;}#chatgpt-mermaid-_r_rc_ .icon-shape p,#chatgpt-mermaid-_r_rc_ .image-shape p{background-color:rgb(252, 252, 252);padding:2px;}#chatgpt-mermaid-_r_rc_ .icon-shape .label rect,#chatgpt-mermaid-_r_rc_ .image-shape .label rect{opacity:0.5;background-color:rgb(252, 252, 252);fill:rgb(252, 252, 252);}#chatgpt-mermaid-_r_rc_ .label-icon{display:inline-block;height:1em;overflow:visible;vertical-align:-0.125em;}#chatgpt-mermaid-_r_rc_ .node .label-icon path{fill:currentColor;stroke:revert;stroke-width:revert;}#chatgpt-mermaid-_r_rc_ .node .neo-node{stroke:rgb(83, 154, 248);}#chatgpt-mermaid-_r_rc_ [data-

look="neo"].node rect,#chatgpt-mermaid-_r_rc_ [data-look="neo"].cluster rect,#chatgpt-mermaid-_r_rc_ [data-look="neo"].node polygon{stroke:url(#chatgpt-mermaid-_r_rc_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rc_ [data-look="neo"].swimlane.cluster rect{filter:none;}#chatgpt-mermaid-_r_rc_ [data-look="neo"].node path{stroke:url(#chatgpt-mermaid-_r_rc_-gradient);stroke-width:1px;}#chatgpt-mermaid-_r_rc_ [data-look="neo"].node .outer-path{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rc_ [data-look="neo"].node .neo-line path{stroke:rgb(83, 154, 248);filter:none;}#chatgpt-mermaid-_r_rc_ [data-look="neo"].node circle{stroke:url(#chatgpt-mermaid-_r_rc_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rc_ [data-look="neo"].node circle .state-start{fill:#000000;}#chatgpt-mermaid-_r_rc_ [data-look="neo"].icon-shape .icon{fill:url(#chatgpt-mermaid-_r_rc_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rc_ [data-look="neo"].icon-shape .icon-neo path{stroke:url(#chatgpt-mermaid-_r_rc_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rc_ .node text{font-size:14px;font-weight:600;letter-spacing:normal;fill:rgb(0, 79, 153);}#chatgpt-mermaid-_r_rc_ .edgeLabels text{font-size:13px;font-weight:600;letter-spacing:-0.08px;fill:rgb(0, 79, 153);}#chatgpt-mermaid-_r_rc_ .node tspan[font-weight="normal"],#chatgpt-mermaid-_r_rc_ .edgeLabels tspan[font-weight="normal"]{font-weight:600;}#chatgpt-mermaid-_r_rc_ .edgeLabel .label rect{opacity:1;rx:13px;ry:13px;fill:rgb(245, 250, 255);stroke:rgb(206, 219, 229);stroke-width:1px;}#chatgpt-mermaid-_r_rc_ .node rect,#chatgpt-mermaid-_r_rc_ .node circle,#chatgpt-mermaid-_r_rc_ .node ellipse,#chatgpt-mermaid-_r_rc_ .node polygon,#chatgpt-mermaid-_r_rc_ .node path{fill:rgb(229, 243, 255);stroke:rgba(0, 0, 0, 0.1);stroke-width:1px;}#chatgpt-mermaid-_r_rc_ .node rect{rx:16px;ry:16px;}#chatgpt-mermaid-_r_rc_ .mermaid-decision .label-container{fill:rgb(245, 250, 255);stroke:rgb(206, 219, 229);stroke-dasharray:2,2;}#chatgpt-mermaid-_r_rc_ .edgePaths .flowchart-link{stroke:rgb(143, 143, 143);stroke-width:1px;stroke-linecap:round;stroke-linejoin:round;}#chatgpt-mermaid-_r_rc_ .marker{fill:rgb(143, 143, 143);stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rc_ :root{--mermaid-font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";}AdvertiserCampaign ManagerAd ServerHouse AdsAd ExchangePaid CampaignsTargeting EnginePublisher InventorySDK AndroidImpressionEvent CollectorFraud DetectionBilling and WalletPublisher RevenueExchange CreditsPlatform Revenue

O Ad Server será responsável por decidir qual anúncio mostrar em cada solicitação.

4.2. Regras de distribuição

Cada impressão poderá disputar espaço entre:

1. Campanhas pagas.
2. Campanhas de troca.
3. House Ads.

O motor deve considerar:

- Segmentação.
- País e região.
- Categoria do aplicativo.
- Formato compatível.
- Orçamento restante.
- Prioridade da campanha.
- Frequência por usuário.
- Limite de impressões.
- Qualidade do publisher.
- Elegibilidade do anúncio.

Eu não utilizaria uma prioridade fixa em que House Ads sempre ganham. Isso prejudicaria a monetização dos publishers.

A seleção deve considerar o rendimento esperado, os compromissos de entrega e a elegibilidade de cada campanha.

4.3. O leilão de publicidade

Na primeira versão, não precisamos construir um leilão complexo em tempo real.

Recomendo um mecanismo de seleção com:

- CPM mínimo do publisher.
- CPM efetivo da campanha.
- CPC máximo do anunciante.
- Probabilidade estimada de clique, quando houver dados suficientes.
- Prioridade contratual.
- Saldo e pacing da campanha.

Uma campanha CPC poderá ser comparada por valor esperado de impressão:

\[eCPM = CPC \times CTR \times 1000 \]

Exemplo:

- CPC: R$ 0,20.
- CTR estimado: 1,5%.
- eCPM estimado: R$ 3,00.

O valor é uma previsão de rendimento, não uma garantia de receita.

4.4. Motor de créditos da troca de anúncios

Este é um componente que precisa de atenção especial.

Não podemos converter simplesmente cada impressão em um crédito fixo. Uma impressão em um aplicativo com público qualificado pode valer mais do que uma impressão de baixa qualidade.

Minha proposta:

\[Créditos = \frac{Impressões\ válidas \times Taxa\ de\ crédito}{1000} \]

A taxa será definida por inventário, formato, qualidade e política comercial.

Exemplo ilustrativo:

Inventário	Impressões válidas	Crédito por mil	Créditos
App Futebol	10.000	1,20	12
App Idiomas	10.000	0,90	9
App Ferramentas	10.000	0,70	7

O valor de cada crédito será uma unidade interna de consumo publicitário, não dinheiro e não um direito automático de saque.

Regras obrigatórias:

- Créditos somente após validação.
- Créditos não podem ser gerados por eventos duplicados.
- Campanhas devem respeitar limites de consumo.
- Saldo e movimentações devem ter histórico imutável.
- A plataforma poderá suspender créditos associados a fraude.
- Créditos promocionais e comprados devem ter saldos separados.

5. SDK Android — Integração com os aplicativos

O SDK é o que permitirá transformar a plataforma em uma rede de verdade.

O desenvolvedor deverá conseguir integrar a publicidade sem precisar implementar todo o motor de anúncios.

Flutter 2.8 更新详解
docs.flutter.cn

5.1. Estrutura do SDK

Nome técnico provisório: `MGP Android SDK`.

Componente	Responsabilidade
MGP Core	Inicialização e configuração
MGP Banner	Exibição de banners
MGP Native	Anúncios nativos
MGP Interstitial	Anúncios de tela cheia

MGP Rewarded	Anúncios recompensados
MGP Consent	Consentimento e privacidade
MGP Events	Eventos e telemetria
MGP Attribution	Atribuição de campanhas

Eu desenvolveria o núcleo nativo em Kotlin e entregaria duas interfaces:

- Android SDK para projetos Kotlin/Java.
- Flutter Plugin para aplicativos Flutter.

Isso permite que sua rede atenda tanto desenvolvedores nativos quanto Flutter sem depender de uma implementação Dart improvisada.

5.2. Experiência de integração

O desenvolvedor receberá:

- App ID.
- SDK Key pública.
- Ad Unit ID.
- Configuração de ambientes.
- Documentação.
- Exemplos de implementação.
- Relatório de integração.

Exemplo conceitual de uso Flutter:

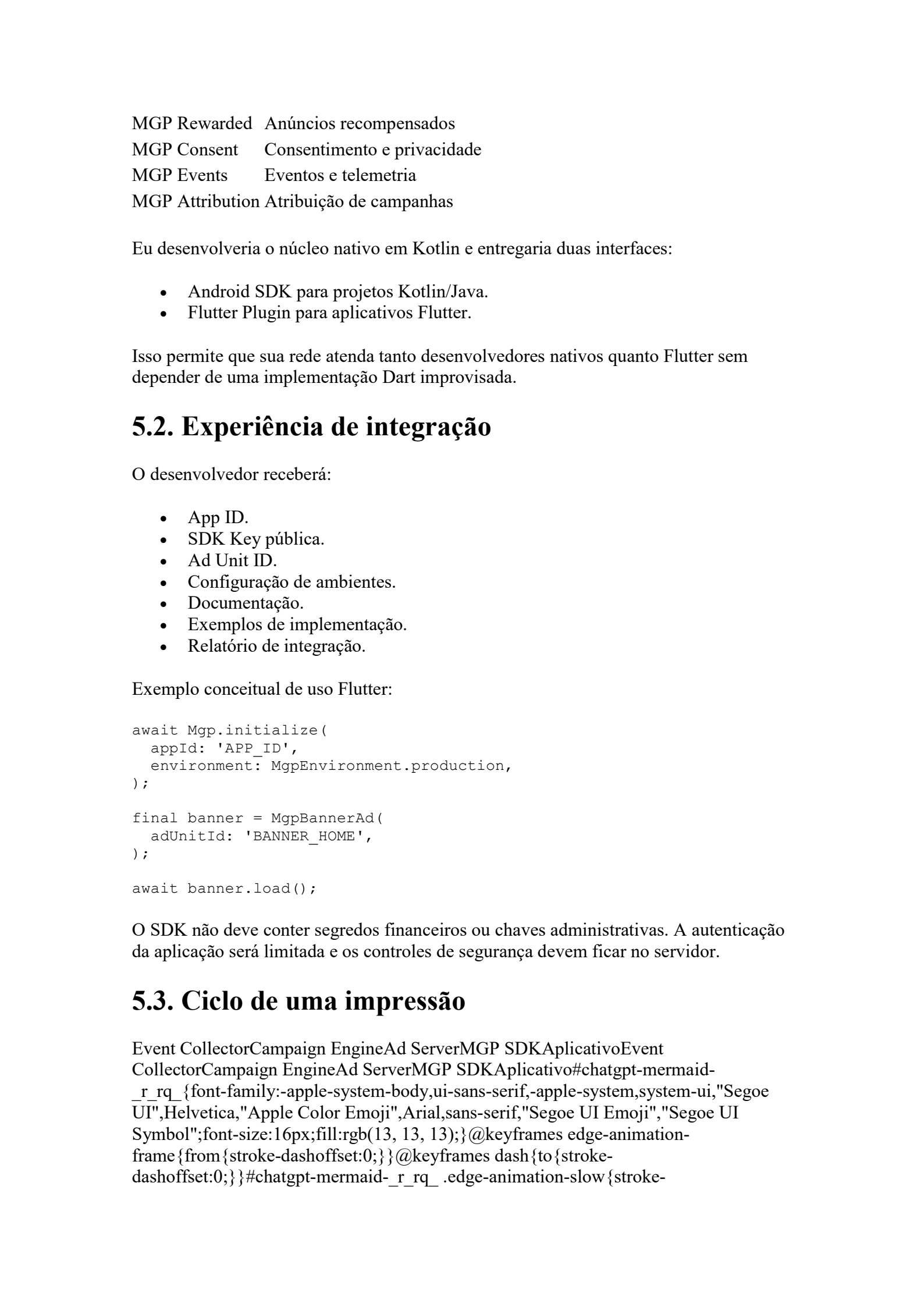
```
await Mgp.initialize(
  appId: 'APP_ID',
  environment: MgpEnvironment.production,
);

final banner = MgpBannerAd(
  adUnitId: 'BANNER_HOME',
);

await banner.load();
```

O SDK não deve conter segredos financeiros ou chaves administrativas. A autenticação da aplicação será limitada e os controles de segurança devem ficar no servidor.

5.3. Ciclo de uma impressão



dasharray:9,5!important;stroke-dashoffset:900;animation:dash 50s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_rq_ .edge-animation-fast{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 20s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_rq_ .error-icon{fill:rgb(243, 243, 243);}#chatgpt-mermaid-_r_rq_ .error-text{fill:rgb(13, 13, 13);stroke:rgb(13, 13, 13);}#chatgpt-mermaid-_r_rq_ .edge-thickness-normal{stroke-width:1px;}#chatgpt-mermaid-_r_rq_ .edge-thickness-thick{stroke-width:3.5px;}#chatgpt-mermaid-_r_rq_ .edge-pattern-solid{stroke-dasharray:0;}#chatgpt-mermaid-_r_rq_ .edge-thickness-invisible{stroke-width:0;fill:none;}#chatgpt-mermaid-_r_rq_ .edge-pattern-dashed{stroke-dasharray:3;}#chatgpt-mermaid-_r_rq_ .edge-pattern-dotted{stroke-dasharray:2;}#chatgpt-mermaid-_r_rq_ .marker{fill:rgb(143, 143, 143);stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rq_ .marker.cross{stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rq_ svg{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;}#chatgpt-mermaid-_r_rq_ p{margin:0;}#chatgpt-mermaid-_r_rq_ .actor{stroke:rgb(83, 154, 248);fill:rgb(222, 234, 251);stroke-width:1;}#chatgpt-mermaid-_r_rq_ rect.actor.outer-path[data-look="neo"]{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rq_ rect.note[data-look="neo"]{stroke:rgb(107, 198, 127);fill:rgb(243, 243, 243);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rq_ text.actor>tspan{fill:rgb(13, 13, 13);stroke:none;}#chatgpt-mermaid-_r_rq_ .actor-line{stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rq_ .innerArc{stroke-width:1.5;stroke-dasharray:none;}#chatgpt-mermaid-_r_rq_ .messageLine0{stroke-width:1.5;stroke-dasharray:none;stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rq_ .messageLine1{stroke-width:1.5;stroke-dasharray:2,2;stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rq_ [id$="-arrowhead"] path{fill:rgb(143, 143, 143);stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rq_ .sequenceNumber{fill:#707070;}#chatgpt-mermaid-_r_rq_ [id$="-sequencenumber"]{fill:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rq_ [id$="-crosshead"] path{fill:rgb(143, 143, 143);stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_rq_ .messageText{fill:rgb(13, 13, 13);stroke:none;}#chatgpt-mermaid-_r_rq_ .labelBox{stroke:rgba(0, 0, 0, 0.1);fill:rgb(252, 252, 252);filter:none;}#chatgpt-mermaid-_r_rq_ .labelText,#chatgpt-mermaid-_r_rq_ .labelText>tspan{fill:rgb(13, 13, 13);stroke:none;}#chatgpt-mermaid-_r_rq_ .loopText,#chatgpt-mermaid-_r_rq_ .loopText>tspan{fill:rgb(13, 13, 13);stroke:none;}#chatgpt-mermaid-_r_rq_ .sectionTitle,#chatgpt-mermaid-_r_rq_ .sectionTitle>tspan{fill:rgb(13, 13, 13);stroke:none;}#chatgpt-mermaid-_r_rq_ .loopLine{stroke-width:2px;stroke-dasharray:2,2;stroke:rgba(0, 0, 0, 0.1);fill:rgba(0, 0, 0, 0.1);}#chatgpt-mermaid-_r_rq_ .note{stroke:rgb(107, 198, 127);fill:rgb(243, 243, 243);}#chatgpt-mermaid-_r_rq_ .noteText,#chatgpt-mermaid-_r_rq_ .noteText>tspan{fill:rgb(13, 13, 13);stroke:none;font-weight:normal;}#chatgpt-mermaid-_r_rq_ .activation0{fill:rgb(243, 243, 243);stroke:hsl(0, 0%, 85.2941176471%);}#chatgpt-mermaid-_r_rq_ .activation1{fill:rgb(243, 243, 243);stroke:hsl(0, 0%, 85.2941176471%);}#chatgpt-mermaid-_r_rq_ .activation2{fill:rgb(243, 243, 243);stroke:hsl(0, 0%, 85.2941176471%);}#chatgpt-mermaid-_r_rq_ .actorPopupMenu{position:absolute;}#chatgpt-mermaid-_r_rq_ .actorPopupMenuPanel{position:absolute;fill:rgb(222, 234, 251);box-shadow:0px 8px 16px 0px rgba(0,0,0,0.2);filter:drop-shadow(3px 5px 2px rgb(0 0 0 / 0.4));}#chatgpt-mermaid-_r_rq_ .actor-man circle,#chatgpt-mermaid-_r_rq_ line{fill:rgb(222, 234, 251);stroke-width:2px;}#chatgpt-mermaid-_r_rq_ g rect.rect{filter:drop-shadow(1px

2px 2px rgba(185,185,185,1));stroke:rgb(83, 154, 248);}#chatgpt-mermaid-_r_rq_ .node .neo-node{stroke:rgb(83, 154, 248);}#chatgpt-mermaid-_r_rq_ [data-look="neo"].node rect,#chatgpt-mermaid-_r_rq_ [data-look="neo"].cluster rect,#chatgpt-mermaid-_r_rq_ [data-look="neo"].node polygon{stroke:url(#chatgpt-mermaid-_r_rq_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rq_ [data-look="neo"].swimlane.cluster rect{filter:none;}#chatgpt-mermaid-_r_rq_ [data-look="neo"].node path{stroke:url(#chatgpt-mermaid-_r_rq_-gradient);stroke-width:1px;}#chatgpt-mermaid-_r_rq_ [data-look="neo"].node .outer-path{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rq_ [data-look="neo"].node .neo-line path{stroke:rgb(83, 154, 248);filter:none;}#chatgpt-mermaid-_r_rq_ [data-look="neo"].node circle{stroke:url(#chatgpt-mermaid-_r_rq_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rq_ [data-look="neo"].node circle .state-start{fill:#000000;}#chatgpt-mermaid-_r_rq_ [data-look="neo"].icon-shape .icon{fill:url(#chatgpt-mermaid-_r_rq_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rq_ [data-look="neo"].icon-shape .icon-neo path{stroke:url(#chatgpt-mermaid-_r_rq_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_rq_ :root{--mermaid-font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";}Solicita anúncioAd RequestSeleciona campanha elegívelCriativo e regrasAd ResponseRenderiza anúncioImpression EventValida eventoConfirmaçãoClick Event (se ocorrer)

Uma impressão só será contabilizada quando atender aos critérios de renderização e visibilidade definidos para seu formato. Receber uma resposta do servidor não significa que houve impressão.

5.4. Segurança do SDK

O SDK precisa ter:

- TLS obrigatório.
- Validação de integridade de requisições.
- Identificação de aplicativo e versão.
- Nonces ou identificadores únicos de evento.
- Proteção contra replay.
- Limites de frequência.
- Detecção de comportamento automatizado.
- Filas locais para eventos temporariamente sem conexão.
- Controle de versão mínima suportada.

O servidor será a autoridade sobre saldo, cobrança e créditos. Nunca confiaremos nos valores calculados exclusivamente no dispositivo.

6. Banco de dados — Modelo de dados central

Eu recomendo PostgreSQL como banco transacional principal, Redis para cache e filas e ClickHouse para grandes volumes de eventos publicitários.

Não colocaria todas as impressões publicitárias no PostgreSQL. Isso pode comprometer o desempenho conforme a rede crescer.

6.1. Organização dos dados

O banco será dividido logicamente por domínios.

PostgreSQL — Dados transacionais

Usuários, empresas, aplicativos, campanhas, pagamentos e carteiras.

ClickHouse — Dados analíticos

Impressões, cliques, requisições, conversões, métricas e séries temporais.

Redis — Dados operacionais

Cache de campanhas, pacing, limites de frequência, filas e configurações.

Object Storage — Arquivos

Criativos, imagens, ícones, relatórios e materiais de campanha.

6.2. Tabelas principais do PostgreSQL

Domínio	Tabelas
Identidade	users, organizations, memberships, roles
Aplicativos	apps, app_versions, app_store_metadata
Inteligência	app_snapshots, rankings, keywords, keyword_history
Concorrência	tracked_apps, competitor_snapshots
Publishers	publisher_accounts, ad_units, inventory_settings
Advertisers	advertiser_accounts, campaigns, ad_groups
Criativos	creatives, creative_assets, creative_reviews
Publicidade	ad_requests, ad_decisions, campaign_delivery
Exchange	exchange_offers, exchange_rules, credit_rates
Carteira	wallets, wallet_accounts, wallet_ledger
Financeiro	invoices, payments, payouts, refunds
Fraude	fraud_signals, invalid_events, publisher_quality
Integrações	integrations, credentials, sync_jobs
SDK	sdk_apps, sdk_keys, sdk_versions
Segurança	audit_logs, consent_records

6.3. Modelo de relacionamento

#chatgpt-mermaid-_r_s5_{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;fill:rgb(13, 13, 13);}@keyframes edge-animation-frame{from{stroke-dashoffset:0;}}@keyframes dash{to{stroke-dashoffset:0;}}#chatgpt-mermaid-_r_s5_ .edge-animation-slow{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 50s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_s5_ .edge-animation-fast{stroke-dasharray:9,5!important;stroke-dashoffset:900;animation:dash 20s linear infinite;stroke-linecap:round;}#chatgpt-mermaid-_r_s5_ .error-icon{fill:rgb(243, 243, 243);}#chatgpt-mermaid-_r_s5_ .error-text{fill:rgb(13, 13, 13);stroke:rgb(13, 13, 13);}#chatgpt-mermaid-_r_s5_ .edge-thickness-normal{stroke-width:1px;}#chatgpt-mermaid-_r_s5_ .edge-thickness-thick{stroke-width:3.5px;}#chatgpt-mermaid-_r_s5_ .edge-pattern-solid{stroke-dasharray:0;}#chatgpt-mermaid-_r_s5_ .edge-thickness-invisible{stroke-width:0;fill:none;}#chatgpt-mermaid-_r_s5_ .edge-pattern-dashed{stroke-dasharray:3;}#chatgpt-mermaid-_r_s5_ .edge-pattern-dotted{stroke-dasharray:2;}#chatgpt-mermaid-_r_s5_ .marker{fill:rgb(143, 143, 143);stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_s5_ .marker.cross{stroke:rgb(143, 143, 143);}#chatgpt-mermaid-_r_s5_ svg{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";font-size:16px;}#chatgpt-mermaid-_r_s5_ p{margin:0;}#chatgpt-mermaid-_r_s5_ .entityBox{fill:rgb(222, 234, 251);stroke:rgb(83, 154, 248);}#chatgpt-mermaid-_r_s5_ .relationshipLabelBox{fill:rgb(243, 243, 243);opacity:0.7;background-color:rgb(243, 243, 243);}#chatgpt-mermaid-_r_s5_ .relationshipLabelBox rect{opacity:0.5;}#chatgpt-mermaid-_r_s5_ .labelBkg{background-color:rgba(243, 243, 243, 0.5);}#chatgpt-mermaid-_r_s5_ .edgeLabel{background-color:rgb(252, 252, 252);}#chatgpt-mermaid-_r_s5_ .edgeLabel .label rect{fill:rgb(252, 252, 252);}#chatgpt-mermaid-_r_s5_ .edgeLabel .label text{fill:rgb(13, 13, 13);}#chatgpt-mermaid-_r_s5_ .edgeLabel .label{fill:rgb(83, 154, 248);font-size:14px;}#chatgpt-mermaid-_r_s5_ .label{font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";color:rgb(13, 13, 13);}#chatgpt-mermaid-_r_s5_ .edge-pattern-dashed{stroke-dasharray:8,8;}#chatgpt-mermaid-_r_s5_ .node rect,#chatgpt-mermaid-_r_s5_ .node circle,#chatgpt-mermaid-_r_s5_ .node ellipse,#chatgpt-mermaid-_r_s5_ .node polygon{fill:rgb(222, 234, 251);stroke:rgb(83, 154, 248);stroke-width:1px;}#chatgpt-mermaid-_r_s5_ .relationshipLine{stroke:rgb(143, 143, 143);stroke-width:1px;fill:none;}#chatgpt-mermaid-_r_s5_ .marker{fill:none!important;stroke:rgb(143, 143, 143)!important;stroke-width:1;}#chatgpt-mermaid-_r_s5_ [data-look=neo].labelBkg{background-color:rgba(243, 243, 243, 0.5);}#chatgpt-mermaid-_r_s5_ .node .neo-node{stroke:rgb(83, 154, 248);}#chatgpt-mermaid-_r_s5_ [data-look="neo"].node rect,#chatgpt-mermaid-_r_s5_ [data-look="neo"].cluster rect,#chatgpt-mermaid-_r_s5_ [data-look="neo"].node polygon{stroke:url(#chatgpt-mermaid-_r_s5_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_s5_ [data-look="neo"].swimlane.cluster rect{filter:none;}#chatgpt-mermaid-_r_s5_ [data-look="neo"].node path{stroke:url(#chatgpt-mermaid-_r_s5_-gradient);stroke-width:1px;}#chatgpt-mermaid-_r_s5_ [data-look="neo"].node .outer-path{filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_s5_ [data-

look="neo"].node .neo-line path{stroke:rgb(83, 154, 248);filter:none;}#chatgpt-mermaid-_r_s5_ [data-look="neo"].node circle{stroke:url(#chatgpt-mermaid-_r_s5_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_s5_ [data-look="neo"].node circle .state-start{fill:#000000;}#chatgpt-mermaid-_r_s5_ [data-look="neo"].icon-shape .icon{fill:url(#chatgpt-mermaid-_r_s5_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_s5_ [data-look="neo"].icon-shape .icon-neo path{stroke:url(#chatgpt-mermaid-_r_s5_-gradient);filter:drop-shadow(1px 2px 2px rgba(185,185,185,1));}#chatgpt-mermaid-_r_s5_ :root{--mermaid-font-family:-apple-system-body,ui-sans-serif,-apple-system,system-ui,"Segoe UI",Helvetica,"Apple Color Emoji",Arial,sans-serif,"Segoe UI Emoji","Segoe UI Symbol";}ORGANIZATIONSMEMBERSHIPSUSERSAPPSAD_UNITSAPP_SNAPSHOTSCAMPAIGNSAD_GROUPSCREATIVESCAMPAIGN_DELIVERYWALLETSWALLET_LEDGEREXCHANGE_OFFERSSDK_APPShasbelongsownsprovidestrackedcreatescontainsusesgeneratesservesownsrecordsfundsintegrates

6.4. Estrutura financeira: carteira e razão contábil

Este ponto não pode ser tratado como uma simples coluna `balance` na tabela de usuários.

A carteira precisa funcionar como um ledger financeiro.

Cada operação gera um lançamento:

- Crédito adquirido.
- Crédito promocional.
- Crédito obtido por troca.
- Crédito consumido.
- Receita de publisher pendente.
- Receita aprovada.
- Receita disponível para saque.
- Ajuste ou estorno.

Eu separaria três saldos:

Carteira	Finalidade
Advertising Credits	Comprar publicidade
Exchange Credits	Trocar inventário
Publisher Earnings	Receita financeira

O saldo será calculado a partir dos lançamentos, com operações transacionais, idempotência e auditoria.

Também manteria contas contábeis separadas para a receita da plataforma, valores devidos aos publishers e valores ainda em validação.

Regra: créditos publicitários não podem ser convertidos automaticamente em dinheiro. Os ganhos financeiros dos publishers seguem outro fluxo, com regras contratuais, tributárias e de pagamento.

7. API da plataforma

A API será construída com NestJS, organizada por módulos e com documentação Swagger/OpenAPI.

O painel web, o SDK Android, o plugin Flutter e integrações externas utilizarão a mesma infraestrutura de serviços.

7.1. Grupos de endpoints

Grupo	Exemplos
Authentication	`/auth/login`, `/auth/register`
Organizations	`/organizations`
Apps	`/apps`, `/apps/:id`
Intelligence	`/intelligence/search`
Rankings	`/rankings`
Keywords	`/keywords/research`
Competitors	`/competitors`
Campaigns	`/campaigns`
Creatives	`/creatives`
Ad Server	`/ads/request`
Publishers	`/publishers`
Ad Units	`/ad-units`
Exchange	`/exchange/offers`
Wallet	`/wallets`
Billing	`/billing`
Payouts	`/payouts`
Analytics	`/analytics`
SDK	`/sdk/config`
Administration	`/admin`

7.2. API pública do Ad Server

Exemplo conceitual de solicitação:

```
{
  "app_id": "app_123",
  "ad_unit_id": "home_banner",
  "format": "banner",
  "placement": "home",
  "country": "BR",
```

```
  "platform": "android",
  "sdk_version": "1.0.0"
}
```

O servidor responde com:

```
{
  "request_id": "req_123",
  "fill": true,
  "campaign_id": "cmp_456",
  "creative": {
    "type": "native",
    "title": "Conheça este aplicativo",
    "description": "Descubra novas funcionalidades",
    "image": "https://cdn.exemplo.com/creative.jpg",
    "click_url": "https://exemplo.com/click/abc"
  },
  "tracking": {
    "impression_token": "signed_token",
    "click_token": "signed_click_token"
  }
}
```

É um exemplo de contrato, não uma resposta de produção. Em produção, cada formato terá um esquema próprio, validação de compatibilidade e URL de destino controlada.

7.3. API de eventos

O SDK enviará eventos para um endpoint de coleta:

```
{
  "event_id": "evt_123",
  "request_id": "req_123",
  "event_type": "impression",
  "app_id": "app_123",
  "ad_unit_id": "home_banner",
  "campaign_id": "cmp_456",
  "timestamp": "2026-10-30T12:00:00Z"
}
```

O servidor valida, deduplica e encaminha o evento ao processamento analítico.

O sistema deverá manter uma separação entre:

- Evento recebido.
- Evento validado.
- Evento faturável.
- Evento atribuído a uma campanha.

Essa separação evita que um clique ou impressão inválida gere cobrança ou créditos.

8. Integrações e fontes de dados

Aqui temos duas áreas distintas: inteligência da Google Play e aquisição publicitária.

8.1. Google Play

Integração	Utilização
Google Play Store	Catálogo e metadados públicos permitidos
Google Play Developer API	Dados dos aplicativos autorizados pelo desenvolvedor
Play Console Reporting API	Qualidade e relatórios dos aplicativos conectados
Reviews API	Avaliações e respostas dos próprios aplicativos
Install Referrer	Atribuição de referências de instalação

Um detalhe importante: a Google Play Developer API não é uma API pública para consultar qualquer aplicativo concorrente. Ela é voltada à administração dos aplicativos das contas de desenvolvedor autorizadas.

Google for Developers+2

Portanto, o catálogo geral de aplicativos precisará de uma estratégia própria de aquisição de dados, respeitando as condições de acesso das fontes, licenciamento e direitos de uso.

Fontes complementares

- Bases comerciais de inteligência mobile.
- Dados públicos disponibilizados pelas lojas.
- Provedores de ASO com licenciamento.
- Informações fornecidas voluntariamente pelos desenvolvedores.
- Histórico de dados coletados e armazenados de forma permitida.

Minha recomendação é começar com uma fonte própria de metadados e rankings, e contratar dados especializados de palavras-chave e estimativas quando houver validação comercial.

8.2. Integrações publicitárias

Integração	Finalidade
Google Play Install Referrer	Atribuição de campanhas
Firebase Analytics	Métricas dos aplicativos participantes
Google Ads	Possível aquisição externa
Meta Ads	Possível aquisição externa
MMP	Atribuição e deduplicação
Mercado Pago ou Pagar.me	Cobranças e pagamentos
E-mail transacional	Avisos e relatórios
CDN	Distribuição de criativos

O Install Referrer permite recuperar informações como referência, horários de clique e instalação inicial, quando a integração e os requisitos são atendidos.

Android Developers+1

Não dependeria de uma atribuição determinística de todas as instalações. O sistema deverá combinar Install Referrer, eventos de primeira abertura, links de campanha e, quando aplicável, mensuração agregada e ferramentas especializadas.

9. Modelo de negócio e monetização

A plataforma terá duas economias paralelas: SaaS e publicidade.

9.1. Receita SaaS

Minha sugestão de planos iniciais:

Plano	Preço mensal	Público
Free	R$ 0	Desenvolvedor iniciante
Starter	R$ 49	Desenvolvedor independente
Pro	R$ 149	Pequena empresa
Business	R$ 399	Software house
Agency	R$ 799	Agência de marketing

Os limites deverão variar conforme o custo real das consultas, aplicativos monitorados, palavras-chave e histórico disponível.

Exemplo de diferenciação

Recurso	Free	Starter	Pro	Business
Apps monitorados	3	10	30	100
Palavras-chave	10	50	200	1.000
Histórico	7 dias	30 dias	90 dias	365 dias
Concorrentes	2	5	15	50
Relatórios	Básicos	Básicos	Avançados	Avançados
API	Não	Não	Limitada	Sim

Esses limites são propostas comerciais iniciais, não limitações técnicas.

9.2. Receita da Ad Network

A plataforma pode ganhar dinheiro de quatro maneiras:

1. Margem sobre publicidade vendida.
2. Comissão sobre campanhas de terceiros.
3. Venda de créditos publicitários.
4. Assinaturas e serviços adicionais.

Exemplo financeiro ilustrativo

Considere uma campanha com 1 milhão de impressões faturáveis a um CPM de R$ 4,00.

Valor pago pelo anunciante

R$ 4.000,00

Publisher (70%)

R$ 2.800

Plataforma (30%)

R$ 1.200

Simulação de divisão comercial antes de custos operacionais, tributos, estornos e ajustes de tráfego inválido.

A participação do publisher pode variar por formato, qualidade do inventário e contrato.

9.3. Troca de publicidade

Na troca, não existe necessariamente dinheiro entrando na operação.

O publisher recebe créditos, que representam direito a consumir inventário publicitário dentro da rede.

Exemplo:

- Publisher disponibiliza inventário.
- A plataforma valida impressões.
- Os créditos são liberados.
- O publisher cria uma campanha.
- A rede consome os créditos à medida que entrega publicidade.

A plataforma pode monetizar indiretamente esse modelo por meio de planos premium, créditos adicionais e serviços pagos.

9.4. House Ads

Eu utilizaria House Ads em dois contextos:

- Divulgação dos aplicativos da própria empresa.
- Campanhas internas da plataforma para promover seus produtos e planos.

Eles terão uma fonte de financiamento e contabilização separada. Não devem gerar uma receita fictícia para o publisher nem consumir orçamento de anunciantes externos sem autorização.

10. Estratégia antifraude e conformidade

Não deixaria isso para uma fase avançada.

A rede publicitária precisa de um núcleo antifraude desde seu primeiro SDK.

Controles técnicos

- Deduplicação de eventos.
- Assinatura de tokens de impressão.
- Validação de timestamps.
- Rate limiting.
- Detecção de cliques anormais.
- Análise de padrões de dispositivos.
- Detecção de tráfego automatizado.
- Monitoramento de publishers.
- Auditoria de campanhas.
- Bloqueio de aplicativos suspeitos.
- Revisão manual de casos de risco.

Privacidade e políticas

O SDK precisará ter uma política de coleta transparente, mecanismo de consentimento quando aplicável, controles de dados, exclusão e retenção.

A LGPD deve ser considerada desde o desenho da plataforma, inclusive para identificadores de publicidade e dados utilizados em segmentação. A ANPD disponibiliza orientações sobre dados empregados em publicidade e identificação de usuários.

Serviços e Informações do Brasil+1

Também precisamos respeitar as regras do Google Play: anúncios fazem parte da experiência do aplicativo, devem ser adequados à classificação do conteúdo e não podem ser invasivos ou interferir no funcionamento normal.

Ajuda do Play Console+1

Uma decisão importante: não utilizaria identificadores permanentes de dispositivo como alternativa ao Advertising ID. O SDK deve respeitar a possibilidade de exclusão e redefinição desse identificador, além das permissões e declarações aplicáveis.

Ajuda do Play Console+1

11. Infraestrutura e implantação

A arquitetura deve aproveitar seu modelo de infraestrutura atual, mas separar a aplicação principal dos componentes de alto volume.

11.1. Estrutura de serviços no EasyPanel

Projeto: MGP Production

Ambiente de produção

mgp-web

Dashboard Next.js

mgp-api

NestJS — API principal

mgp-adserver

Entrega de anúncios e seleção de campanhas

mgp-workers

Coleta, sincronização e processamento

mgp-postgres

Banco transacional

mgp-redis

Cache e filas

mgp-clickhouse

Analytics publicitário

mgp-admin

Administração e monitoramento

Essa é a separação lógica inicial. Não significa que todos precisam ser implantados como containers independentes no primeiro dia.

Ambientes

- Desenvolvimento.
- Homologação.
- Produção.

Todos com bancos, credenciais, filas e configurações separados.

Domínios sugeridos

Domínio	Serviço
`mgp.com.br`	Site comercial
`app.mgp.com.br`	Dashboard
`api.mgp.com.br`	API
`ads.mgp.com.br`	Ad Server
`cdn.mgp.com.br`	Criativos
`docs.mgp.com.br`	Documentação
`status.mgp.com.br`	Status da plataforma

São sugestões de nomenclatura; a disponibilidade dos domínios precisará ser verificada.

Atenção à infraestrutura: eu não dimensionaria o projeto completo para um único VPS pequeno. O MVP pode começar em um VPS com serviços reduzidos, mas Ad Server, banco e processamento de eventos precisam permitir escalabilidade independente. ClickHouse pode ser introduzido quando o volume justificar seu custo operacional.

12. Roadmap de desenvolvimento

O projeto será executado em 10 fases principais. Cada fase terá implementação, testes, auditoria e aprovação antes de avançar.

Minha recomendação é não tentar construir o AppTweak inteiro antes de lançar a primeira rede de publicidade. Os dois produtos devem evoluir em paralelo, mas com entregas menores e integradas.

Roadmap MGP

Estimativas de planejamento, considerando desenvolvimento incremental com equipe pequena e apoio de agentes de programação.

01

Fundação SaaS

2–3 semanas

Autenticação, organizações, permissões, banco, infraestrutura e painel inicial.

02

App Intelligence MVP

3–5 semanas

Catálogo Google Play, rankings, pesquisa, avaliações e concorrentes.

03

ASO Studio

3–4 semanas

Palavras-chave, histórico, auditorias e oportunidades.

04

Publisher & SDK

3–5 semanas

Cadastro de aplicativos, inventário, SDK Kotlin e Flutter, banner e native.

05

House Ads

2–3 semanas

Campanhas próprias, distribuição e relatórios.

06

Ad Exchange

3–5 semanas

Troca de inventário, créditos, regras de conversão e antifraude.

07

Paid Ad Network

4–6 semanas

Campanhas pagas, CPM, CPC, cobrança, pacing e repasses.

08

Monetization Hub

2–4 semanas

Carteiras, faturamento, pagamentos, saques e relatórios financeiros.

09

Analytics & Fraud

3–4 semanas

ClickHouse, análises avançadas, atribuição e detecção de fraude.

10

Lançamento comercial

2–3 semanas

Homologação, documentação, planos, suporte e entrada de publishers.

27–42 semanas

Estimativa sequencial de referência. Algumas fases podem ser parcialmente paralelizadas, mas integrações e homologações podem alterar o prazo.

12.1. Critérios de aprovação por fase

Não avançaremos apenas porque o código foi escrito.

Cada fase deverá entregar:

1. Código versionado no GitHub.
2. Deploy funcional em homologação.
3. Testes automatizados.
4. Testes de integração.
5. Verificação de segurança.
6. Relatório técnico do Antigravity.
7. Auditoria independente das funcionalidades.
8. Lista de pendências e riscos.
9. Aprovação antes da próxima fase.

Isso segue seu fluxo de trabalho preferido: Antigravity executa, ChatGPT atua como gerente técnico e auditor.

13. MVP comercial: o que precisa estar pronto para lançar?

Apesar do escopo completo, eu estabeleceria um primeiro lançamento comercial menor.

Primeiro lançamento público

Produto	Funcionalidade mínima
App Intelligence	Pesquisa, rankings e concorrentes
ASO	Pesquisa básica de palavras-chave
Publisher	Cadastro de aplicativos e inventário
SDK	Banner e Native Ads
House Ads	Campanhas internas
Ad Exchange	Troca de impressões por créditos
Wallet	Saldo e histórico de créditos
Analytics	Impressões, cliques e CTR
Admin	Aprovação e controle de publishers

O primeiro lançamento não precisa oferecer CPI, vídeo recompensado, atribuição avançada ou inteligência competitiva sofisticada.

Esses recursos entrarão após a validação do funcionamento básico.

14. Estratégia para conseguir os primeiros publishers

Uma rede publicitária sem inventário e sem anunciantes não tem valor. Precisamos resolver o problema de início de mercado.

Eu usaria seu próprio portfólio de aplicativos como primeiro grupo de publishers.

Etapa 1 — Inventário próprio

Integrar o SDK em aplicativos próprios:

- Aplicativo de futebol.
- Aplicativos de idiomas.
- Aplicativos utilitários.
- Outros aplicativos publicados.

Isso permite validar a distribuição de House Ads e a troca de publicidade em um ambiente controlado.

Etapa 2 — Desenvolvedores externos

Abrir cadastro gratuito para desenvolvedores brasileiros.

Oferecer:

- SDK gratuito.
- Campanhas House Ads.
- Créditos iniciais promocionais.
- Painel de métricas.
- Plano gratuito de inteligência.
- Ferramentas de cross-promotion.

Etapa 3 — Anunciantes

Buscar:

- Desenvolvedores de aplicativos.
- Software houses.
- Agências.
- Empresas digitais.
- Marcas interessadas em públicos mobile.

Etapa 4 — Escala

Somente depois de demonstrar inventário real, qualidade de tráfego, entrega e resultados, ampliar a venda de publicidade.

15. Indicadores de sucesso da plataforma

O painel administrativo deverá ter indicadores operacionais e comerciais.

Indicador	O que mede
Apps cadastrados	Tamanho do catálogo
Publishers ativos	Oferta publicitária
Advertisers ativos	Demanda publicitária
DAU dos apps publishers	Audiência potencial
Ad Requests	Solicitações de publicidade
Fill Rate	Taxa de preenchimento
Valid Impressions	Impressões válidas
CTR	Taxa de cliques
eCPM	Receita efetiva por mil impressões
Exchange Credits	Volume de troca
Revenue	Receita da rede
Publisher Payout	Valores devidos
Invalid Traffic Rate	Qualidade do tráfego
Campaign ROI	Retorno das campanhas, quando mensurável

Também criaria um painel de saúde da plataforma, acompanhando:

- Disponibilidade do Ad Server.
- Latência de entrega.
- Erros do SDK.
- Falhas de sincronização.
- Eventos pendentes.
- Filas atrasadas.
- Consistência das carteiras.
- Divergência entre impressões entregues e faturadas.

16. Principais riscos e decisões técnicas

Risco	Tratamento
Falta de dados de buscas da Play Store	Estimativas e fornecedores especializados
Pouco inventário publicitário	Portfólio próprio e aquisição de publishers
Fraude de impressões	Validação e antifraude desde o SDK
Pouca demanda de anunciantes	House Ads e troca como entrada
Custo elevado de analytics	ClickHouse apenas quando justificado
Problemas de atribuição	Install Referrer e integração especializada
Dependência de fontes externas	Camada de provedores intercambiáveis
Rejeição de aplicativos	Compliance de anúncios e privacidade

Um ponto adicional: não utilizaria o Privacy Sandbox para Android como fundamento obrigatório da arquitetura, pois a documentação do Google registra sua descontinuação em outubro de 2025. O sistema deverá manter uma estratégia de atribuição compatível com as APIs e políticas efetivamente suportadas.

Google for Developers+1

17. Organização do projeto no GitHub

Sugiro um monorepo para manter os componentes relacionados, com possibilidade de extração futura.

```
mgp-platform/
│
├── apps/
│   ├── web/                  # Dashboard Next.js
│   ├── api/                  # NestJS principal
│   ├── ad-server/            # Motor de anúncios
│   ├── workers/              # Processamento assíncrono
│   └── admin/                # Administração
│
├── packages/
│   ├── contracts/            # Contratos e DTOs
│   ├── database/             # Prisma e migrations
│   ├── shared/               # Tipos e utilitários
│   └── config/               # Configurações compartilhadas
```

```
│
├── sdk/
│   ├── android/              # SDK Kotlin
│   └── flutter/              # Plugin Flutter
│
├── infrastructure/
│   ├── docker/
│   ├── easypanel/
│   └── monitoring/
│
├── docs/
│   ├── architecture/
│   ├── database/
│   ├── api/
│   ├── sdk/
│   ├── security/
│   └── business/
│
└── README.md
```

Stack definitiva proposta

Camada	Tecnologia
Frontend	Next.js + TypeScript
Backend	NestJS + TypeScript
ORM	Prisma
Banco	PostgreSQL
Analytics	ClickHouse
Cache	Redis
Filas	BullMQ
SDK Android	Kotlin
SDK Flutter	Dart
Mobile	Android
Infraestrutura	Docker + EasyPanel
Versionamento	GitHub
Documentação API	Swagger
Monitoramento	Prometheus + Grafana

18. Minha recomendação final de execução

Eu faria uma escolha importante: não começaria pelo SDK nem pelo motor publicitário isoladamente.

A ordem correta é construir primeiro a identidade dos aplicativos, publishers, anunciantes, campanhas e inventário. Depois o Ad Server poderá operar sobre essas estruturas.

Também manteria a inteligência de mercado independente da rede publicitária, mas com integração entre elas. Isso permitirá que a plataforma continue funcionando mesmo se um fornecedor de dados ou publicidade for substituído.

Primeiro marco de desenvolvimento

Fase 1 — Fundação SaaS

Objetivo: criar uma base sólida para todos os módulos futuros.

Entregas:

- Monorepo e configuração do GitHub.
- NestJS e Next.js.
- PostgreSQL e Prisma.
- Redis.
- Autenticação.
- Organizações e multi-tenancy.
- Controle de permissões.
- Painel administrativo inicial.
- Infraestrutura EasyPanel.
- CI/CD.
- Logs e monitoramento.
- Documentação técnica.

Critério de conclusão: um desenvolvedor consegue criar uma conta, cadastrar uma organização, acessar seu painel e administrar seus aplicativos sem acessar dados de outras organizações.

19. Como vamos conduzir a implementação

Para cada fase, prepararemos um prompt específico para Antigravity com:

- Objetivo e escopo.
- Estrutura de arquivos.
- Regras de implementação.
- Banco de dados.
- Endpoints.
- Requisitos de segurança.
- Testes obrigatórios.
- Critérios de aceite.
- Proibições de alterações desnecessárias.
- Relatório técnico final.

Não vamos permitir que o agente invente funcionalidades, substitua tecnologias sem autorização ou reescreva módulos que já estejam funcionando.

Minha sugestão é transformar este documento na especificação-mestre do MGP e iniciar pela Fase 1. As próximas fases deverão obedecer a essa especificação, com alterações documentadas quando houver necessidade.

O objetivo é construir não apenas um painel de análise, mas uma empresa de tecnologia com três ativos estratégicos: uma base de inteligência de aplicativos, uma rede própria de distribuição publicitária e uma infraestrutura de monetização para desenvolvedores Android.

Vamos organizar o desenvolvimento do MGP — Mobile Growth Platform em etapas, cada uma contendo 10 fases. Cada fase terá um objetivo específico, entregáveis, critérios de aceite e auditoria técnica antes de avançarmos.

A primeira etapa será dedicada à fundação da plataforma. Não vamos implementar ainda os módulos completos de inteligência de aplicativos ou publicidade. Vamos preparar a estrutura para que eles possam ser adicionados sem retrabalho.

ETAPA 1 — Fundação e infraestrutura da plataforma

10 fases planejadas
Base de todo o projeto

Objetivo: construir a infraestrutura técnica, arquitetura, segurança, banco de dados central e ambiente de desenvolvimento do MGP.

Etapa 1 — Visão geral

Progresso

0/10 fases

Nenhuma fase iniciada.

01

Planejamento técnico e especificação

Arquitetura, requisitos e decisões técnicas.

Pendente

02

Configuração do ambiente e infraestrutura

VPS, EasyPanel, Docker e ambientes.

Pendente

03

Monorepo e estrutura do projeto

GitHub, organização e padrões de código.

Pendente

04

Backend central NestJS

API, módulos e configurações.

Pendente

05

Banco de dados e modelo multi-tenant

PostgreSQL, Prisma e estrutura central.

Pendente

06

Autenticação e segurança

Login, tokens, proteção e recuperação de conta.

Pendente

07

Organizações, usuários e permissões

Empresas, membros, papéis e RBAC.

Pendente

08

Frontend e dashboard principal

Next.js, layout e navegação.

Pendente

09

Administração e monitoramento

Painel administrativo, logs e saúde do sistema.

Pendente

10

Integração, testes e auditoria final

Deploy, testes, documentação e validação.

Pendente

FASE 01 — Planejamento técnico e especificação

Fundação documental

Objetivo: definir exatamente como o sistema será construído antes de iniciar a programação.

Entregáveis:

1. Documento de arquitetura geral.
2. Definição dos módulos e suas responsabilidades.
3. Stack tecnológica oficial.
4. Padrão de nomenclatura.
5. Padrão de organização dos arquivos.
6. Estratégia multi-tenant.
7. Padrão de API REST.
8. Estratégia de segurança.
9. Padrão de logs e auditoria.
10. Critérios de qualidade e aprovação.

Critério de aceite: arquitetura documentada, coerente e aprovada, sem decisões técnicas críticas pendentes.

FASE 02 — Configuração do ambiente e infraestrutura

Objetivo: preparar o ambiente de hospedagem e execução.

Entregáveis:

- Configuração do Hostinger VPS.
- EasyPanel.
- Docker.
- Configuração de rede.
- Domínios e subdomínios.
- Certificados SSL.
- Variáveis de ambiente.
- Ambientes Development, Staging e Production.
- Política de backups.
- Configuração de recursos e limites dos serviços.

Critério de aceite: ambiente acessível, protegido, documentado e pronto para receber os serviços.

FASE 03 — Monorepo e estrutura do projeto

Objetivo: criar a estrutura central de desenvolvimento.

Entregáveis:

- Repositório GitHub `mgp-platform`.
- Monorepo.
- Estrutura de diretórios.
- Configuração TypeScript.
- Gerenciador de pacotes.
- ESLint e Prettier.
- Convenções de commits.
- Configuração de branches.
- GitHub Actions.
- Documentação inicial.

Estrutura prevista:

```
mgp-platform/
├── apps/
│   ├── api/
│   ├── web/
│   ├── admin/
│   ├── ad-server/
│   └── workers/
├── packages/
│   ├── shared/
│   ├── contracts/
│   ├── database/
│   └── config/
├── sdk/
│   ├── android/
│   └── flutter/
```

```
├── infrastructure/
├── docs/
└── README.md
```

Critério de aceite: repositório organizado, aplicações inicializadas e pipeline de validação funcionando.

FASE 04 — Backend central NestJS

Objetivo: construir o núcleo da API.

Entregáveis:

- Aplicação NestJS.
- Configuração global.
- Configuração de ambiente.
- Módulo de saúde (`HealthModule`).
- Módulo de configuração.
- Módulo de logs.
- Tratamento global de exceções.
- Validação de dados.
- Swagger.
- Versionamento da API.

Estrutura inicial:

```
apps/api/src/
├── main.ts
├── app.module.ts
├── common/
├── config/
├── database/
├── modules/
│   ├── health/
│   ├── auth/
│   ├── users/
│   ├── organizations/
│   └── admin/
└── infrastructure/
```

Critério de aceite: API inicial executando, documentação acessível e endpoints básicos respondendo corretamente.

FASE 05 — Banco de dados e modelo multi-tenant

Objetivo: preparar a base de dados que atenderá todos os módulos futuros.

Tecnologias: PostgreSQL + Prisma.

Entregáveis:

- Configuração de conexão.

- Prisma Schema.
- Migrations.
- Usuários.
- Organizações.
- Membresias.
- Papéis.
- Permissões.
- Sessões.
- Logs de auditoria.
- Isolamento de dados por organização.

Critério de aceite: migrations executadas, relacionamentos validados e testes demonstrando que uma organização não consegue acessar os dados de outra.

FASE 06 — Autenticação e segurança

Objetivo: implementar o sistema de identidade.

Entregáveis:

- Cadastro.
- Login.
- Logout.
- Access Token.
- Refresh Token com rotação.
- Recuperação de senha.
- Verificação de e-mail.
- Controle de sessões.
- Proteção contra tentativas excessivas.
- Gestão de dispositivos e sessões ativas.

Também serão definidos os padrões de segurança das futuras chaves de API e do SDK.

Critério de aceite: fluxos de autenticação testados, sessões protegidas e tokens inválidos ou revogados rejeitados.

FASE 07 — Organizações, usuários e permissões

Objetivo: implementar a estrutura empresarial da plataforma.

Uma conta poderá participar de uma ou mais organizações.

Papéis iniciais:

Papel	Permissões
Owner	Controle total da organização
Admin	Administração
Manager	Gestão operacional

Analyst	Consulta e relatórios
Developer	Aplicativos e integrações
Advertiser	Campanhas publicitárias
Publisher	Inventário e monetização

Um mesmo usuário poderá ter mais de um papel, conforme a organização.

Critério de aceite: criação de organizações, convites, membros, permissões e isolamento de acesso funcionando.

FASE 08 — Frontend e dashboard principal

Objetivo: construir a interface de utilização do MGP.

Tecnologia: Next.js + TypeScript.

BreezeBI
breeze.bi

Entregáveis:

- Tela de login.
- Cadastro.
- Recuperação de senha.
- Dashboard principal.
- Menu lateral.
- Seletor de organização.
- Perfil do usuário.
- Configurações.
- Gestão de membros.
- Layout responsivo.

Menu inicial:

```
MGP Dashboard
│
├── Visão Geral
├── App Intelligence
├── ASO Studio
├── Ad Network
├── Campaigns
├── Publishers
├── Advertisers
├── Monetization
├── Analytics
│
├── Minha Organização
├── Configurações
└── Ajuda
```

Nesta fase, as áreas futuras poderão aparecer como módulos em preparação, mas não terão funcionalidades fictícias.

Critério de aceite: navegação funcional, autenticação integrada e interface conectada à API.

FASE 09 — Administração e monitoramento

Objetivo: permitir que o proprietário da plataforma controle sua operação.

Entregáveis:

- Painel administrativo.
- Gestão de usuários.
- Gestão de organizações.
- Bloqueio e desbloqueio de contas.
- Visualização de logs.
- Histórico de atividades.
- Monitoramento da API.
- Monitoramento do banco.
- Monitoramento do Redis.
- Alertas de erros.

Critério de aceite: administração central funcional, com ações autorizadas e auditadas.

FASE 10 — Integração, testes e auditoria final

Objetivo: verificar se toda a fundação está preparada para receber as próximas etapas.

Entregáveis:

- Testes unitários.
- Testes de integração.
- Testes de API.
- Testes de frontend.
- Testes de isolamento multi-tenant.
- Testes de segurança.
- Testes de deploy.
- Verificação de backups.
- Documentação técnica.
- Relatório final do Antigravity.
- Auditoria de arquitetura.

Critério de aceite: todos os fluxos fundamentais aprovados, sem falhas críticas ou vulnerabilidades conhecidas pendentes.

20. Regras de execução da Etapa 1

Estas regras serão aplicadas a todas as fases:

1. Uma fase por vez.
2. Antigravity executa a implementação.
3. Não alterar funcionalidades já aprovadas sem necessidade.
4. Não inventar funcionalidades.
5. Não antecipar módulos de etapas posteriores.
6. Não utilizar dados fictícios como se fossem dados reais.
7. Não deixar botões ou funções sem implementação.
8. Testes obrigatórios antes da aprovação.
9. Entregar relatório técnico ao final de cada fase.
10. Só iniciar a próxima fase após minha auditoria e aprovação.

21. Controle de progresso

Etapa 1 — Acompanhamento

0/10 concluídas

Fase 01 — Planejamento técnico e especificação

Fase 02 — Configuração do ambiente e infraestrutura

Fase 03 — Monorepo e estrutura do projeto

Fase 04 — Backend central NestJS

Fase 05 — Banco de dados e modelo multi-tenant

Fase 06 — Autenticação e segurança

Fase 07 — Organizações, usuários e permissões

Fase 08 — Frontend e dashboard principal



Fase 09 — Administração e monitoramento



Fase 10 — Integração, testes e auditoria final

Reiniciar acompanhamento

Controle visual de referência. A aprovação técnica continua sendo registrada nos relatórios de execução.

Próximo passo: Fase 01 — Planejamento técnico e especificação.

Vamos começar criando o primeiro prompt completo para o Antigravity, com instruções para produzir a documentação técnica oficial do MGP, sem iniciar a implementação prematuramente.